Python Methods Cheat Sheet

📝 STRING METHODS

Case Conversion

python

```python
text = "Hello World"

text.upper()            # "HELLO WORLD"
text.lower()            # "hello world"
text.capitalize()       # "Hello world"
text.title()            # "Hello World"
text.swapcase()         # "hELLO wORLD"
```

Whitespace & Trimming

python

```python
text = "  hello  "

text.strip()            # "hello" (removes both sides)
text.lstrip()           # "hello  " (removes left)
text.rstrip()           # "  hello" (removes right)
text.replace(" ", "")   # "hello" (removes all spaces)
```

Searching & Checking

python

```python
text = "Hello World"

text.find("World")      # 6 (index position, -1 if not found)
text.index("World")     # 6 (raises error if not found)
text.count("o")         # 2 (counts occurrences)
text.startswith("He")   # True
text.endswith("ld")     # True
"World" in text         # True
```

Validation Methods

python

```
"hello".isalpha()       # True (only letters)
"123".isdigit()         # True (only digits)
"hello123".isalnum()    # True (letters + digits)
"   ".isspace()         # True (only whitespace)
"Hello".islower()       # False
"HELLO".isupper()       # True
"Hello World".istitle()# True
```

Splitting & Joining

python

```
text = "apple,banana,orange"

text.split(",")         # ['apple', 'banana', 'orange']
text.split()            # splits by whitespace
"hello".split("")       # Error! Can't split by empty string

# Joining
fruits = ["apple", "banana"]
", ".join(fruits)       # "apple, banana"
"".join(fruits)         # "applebanana"
```

Replacing & Formatting

python

```
text = "Hello World"

text.replace("World", "Python")  # "Hello Python"
text.replace("l", "L", 1)        # "HeLlo World" (replace only 1)

# Formatting
name = "John"
age = 25
f"Name: {name}, Age: {age}"      # "Name: John, Age: 25"
"Name: {}, Age: {}".format(name, age)  # Same result
```

Alignment & Padding

python

```
text = "hello"

text.center(10)         # "  hello   "
text.center(10, "*")    # "**hello***"
```

```
text.ljust(10)          # "hello     " (left justify)
text.rjust(10)          # "     hello" (right justify)
text.zfill(10)          # "00000hello" (zero padding)
```

📋 LIST METHODS

Adding Elements

```python
fruits = ["apple", "banana"]

fruits.append("orange")         # Adds to end: ['apple', 'banana', 'orange']
fruits.insert(1, "mango")       # Inserts at index: ['apple', 'mango', 'banana']
fruits.extend(["grape", "kiwi"]) # Adds multiple: ['apple', 'banana', 'grape', 'kiwi']
```

Removing Elements

```python
fruits = ["apple", "banana", "orange", "banana"]

fruits.remove("banana")  # Removes first occurrence: ['apple', 'orange', 'banana']
fruits.pop()             # Removes & returns last: 'banana'
fruits.pop(0)            # Removes & returns at index 0: 'apple'
fruits.clear()           # Removes all: []
del fruits[0]            # Deletes element at index 0
```

Searching & Counting

```python
fruits = ["apple", "banana", "orange", "banana"]

fruits.index("banana")   # 1 (first occurrence)
fruits.count("banana")   # 2
"apple" in fruits        # True
```

Sorting & Reversing

```python
```

```python
numbers = [3, 1, 4, 1, 5]

numbers.sort()            # [1, 1, 3, 4, 5] (modifies original)
numbers.sort(reverse=True) # [5, 4, 3, 1, 1] (descending)
sorted(numbers)           # Returns new sorted list (original
unchanged)

numbers.reverse()         # [5, 1, 4, 1, 3] (reverses in place)
numbers[::-1]             # Returns reversed copy
```

Copying

```python
original = [1, 2, 3]

copy1 = original.copy()   # Shallow copy
copy2 = original[:]       # Shallow copy (slice)
copy3 = list(original)    # Shallow copy
```

Other Useful Methods

```python
numbers = [1, 2, 3, 4, 5]

len(numbers)              # 5 (length)
min(numbers)              # 1
max(numbers)              # 5
sum(numbers)              # 15
```

🔒 TUPLE METHODS

Basic Methods (Tuples are immutable!)

```python
numbers = (1, 2, 3, 2, 4)

numbers.count(2)          # 2 (counts occurrences)
numbers.index(3)          # 2 (first occurrence index)
len(numbers)              # 5
```

Tuple Operations

```python
tuple1 = (1, 2)
tuple2 = (3, 4)

tuple1 + tuple2          # (1, 2, 3, 4) (concatenation)
tuple1 * 3               # (1, 2, 1, 2, 1, 2) (repetition)
2 in tuple1              # True (membership)
```

Converting

```python
my_tuple = (1, 2, 3)

list(my_tuple)           # [1, 2, 3] (tuple to list)
tuple([1, 2, 3])         # (1, 2, 3) (list to tuple)
```

📖 DICTIONARY METHODS

Accessing Values

```python
student = {"name": "John", "age": 20, "grade": "A"}

student["name"]          # "John" (KeyError if not exists)
student.get("name")      # "John" (returns None if not exists)
student.get("city", "Unknown")  # "Unknown" (default value)
```

Adding/Updating

```python
student = {"name": "John"}

student["age"] = 20      # Adds new key-value
student.update({"grade": "A", "age": 21})  # Updates multiple
student.setdefault("city", "Dhaka")  # Adds only if key doesn't exist
```

Removing

```python
student = {"name": "John", "age": 20, "grade": "A"}

student.pop("age")       # Removes & returns: 20
```

```
student.pop("city", "Not found")  # Returns default if key missing
student.popitem()         # Removes & returns last item: ('grade', 'A')
del student["name"]       # Deletes key
student.clear()           # Removes all: {}
```

Keys, Values, Items

```python
student = {"name": "John", "age": 20, "grade": "A"}

student.keys()            # dict_keys(['name', 'age', 'grade'])
student.values()          # dict_values(['John', 20, 'A'])
student.items()           # dict_items([('name', 'John'), ('age', 20),
('grade', 'A')])

# Iterating
for key in student.keys():
    print(key)

for value in student.values():
    print(value)

for key, value in student.items():
    print(f"{key}: {value}")
```

Copying & Merging

```python
original = {"a": 1, "b": 2}

copy = original.copy()    # Shallow copy

dict1 = {"a": 1}
dict2 = {"b": 2}
dict1.update(dict2)       # Merges dict2 into dict1
```

Checking

```python
student = {"name": "John", "age": 20}

"name" in student         # True (checks if key exists)
"John" in student.values()  # True (checks if value exists)
```

```python
len(student)              # 2
```

📁 FILE METHODS

Opening Files

```python
# Basic syntax: open(filename, mode, encoding)

file = open("file.txt", "r")            # Read mode
file = open("file.txt", "w")            # Write mode (overwrites)
file = open("file.txt", "a")            # Append mode
file = open("file.txt", "r+")           # Read + Write
file = open("file.txt", "rb")           # Read binary
file = open("file.txt", "r", encoding="utf-8")  # With encoding
```

Reading Methods

```python
with open("file.txt", "r", encoding="utf-8") as file:

    content = file.read()           # Reads entire file as string
    content = file.read(10)         # Reads first 10 characters

    line = file.readline()          # Reads one line
    lines = file.readlines()        # Reads all lines into list

    # Best practice: iterate directly
    for line in file:
        print(line.strip())
```

Writing Methods

```python
with open("file.txt", "w", encoding="utf-8") as file:

    file.write("Hello World")       # Writes string (no newline)
    file.write("Line 2\n")          # Add \n for newline

    lines = ["Line 1\n", "Line 2\n"]
    file.writelines(lines)          # Writes list of strings
```

File Position & Info

```python
with open("file.txt", "r") as file:

    file.tell()                     # Returns current position
    file.seek(0)                    # Moves to start of file
    file.seek(10)                   # Moves to position 10

    file.name                       # Returns filename
    file.mode                       # Returns mode ('r', 'w', etc.)
    file.closed                     # Returns True if closed
```

File Operations (using os module)

```python
import os

os.path.exists("file.txt")          # True if exists
os.path.isfile("file.txt")          # True if file
os.path.isdir("folder")             # True if directory
os.path.getsize("file.txt")         # Size in bytes
os.remove("file.txt")               # Deletes file
os.rename("old.txt", "new.txt")     # Renames file

# Directory operations
os.mkdir("folder")                  # Creates directory
os.rmdir("folder")                  # Removes empty directory
os.listdir(".")                     # Lists files in directory
os.getcwd()                         # Current working directory
```

🎯 QUICK REFERENCE TABLE

Type	Mutable?	Common Methods
String	No	`.upper()`, `.lower()`, `.strip()`, `.split()`, `.replace()`, `.find()`

List	Yes	`.append()`, `.remove()`, `.pop()`, `.sort()`, `.reverse()`, `.extend()`
Tuple	No	`.count()`, `.index()`
Dictionary	Yes	`.get()`, `.keys()`, `.values()`, `.items()`, `.pop()`, `.update()`
File	-	`.read()`, `.readline()`, `.readlines()`, `.write()`, `.close()`

💡 PRO TIPS

1. Always use `with` for files - automatically closes files
2. Use `.strip()` when reading files - removes extra whitespace
3. Use `encoding="utf-8"` for non-English text - handles Bangla, Arabic, etc.
4. Lists are mutable, tuples are not - use tuples for fixed data
5. Use `.get()` for dictionaries - avoids KeyError
6. Use f-strings for formatting - cleaner than `.format()`

🔄 CONVERSION CHEAT SHEET

python

```python
# String to List
"hello".split("")        # Error!
list("hello")            # ['h', 'e', 'l', 'l', 'o']

# List to String
"".join(['h', 'e'])      # "he"

# String to Int/Float
int("123")               # 123
float("12.5")            # 12.5

# Int/Float to String
str(123)                 # "123"

# List/Tuple conversion
list((1, 2, 3))          # [1, 2, 3]
tuple([1, 2, 3])         # (1, 2, 3)
```